

BİL 401

PROJE ARA RAPORU

(Status Report)

Nisa Naz Keleşoğlu
İclal Zeynep Kaya
Zeynep Saçaklı

1. Mevcut Durum

Projenin bu aşaması itibarıyla veri toplama ve veriyi analize hazır hale getirme (ingestion & preprocessing) süreçleri başarıyla tamamlanmıştır. Veri toplama fazı sadece ham verilerin indirilmesini değil, bu verilerin projenin "Zaman ve Maliyet Tahminleme" (Cost & Time Estimation) amacına uygun olarak optimize edilmesini de kapsamaktadır.

1.1 Ham Veri İndirme

- **Veri Kaynağı:** New York City Taxi & Limousine Commission (TLC) resmi portalı üzerinden 2023 yılının tamamına ait resmi kayıtlar hedef alınmıştır.
- **Veri Hacmi:** 12 adet ham Parquet dosyasından oluşan, toplamda yaklaşık 606 MB büyüklüğünde ve ~38.3 milyon satır barındıran bir veri kümesi yerel diske (data/raw/) başarıyla indirilmiştir. Parquet formatının GZIP sıkıştırması ve kolonar yapısı sayesinde disk üzerinde görece küçük yer tutan bu veri, ham CSV eşdeğerinde yaklaşık 5-8 GB'a; bellekte açılmış halinde ise daha da büyük bir alana karşılık gelmektedir. Satır sayısı ve bellek yükü açısından bu veri seti, tek makine ölçeğinde 'orta-büyük' bantta değerlendirilebilecek bir ölçek sunmaktadır. Bu bilinçli tercih, iki motorun avantaj bölgelerinin çakıştığı 'gri bölgeyi' inceleyebilmemize olanak tanımaktadır; petabyte ölçeğinde bir cluster senaryosunda Spark'ın belirgin üstünlüğü karşılaştırmayı tek yönlü hale getirecektir.

1.2 Data Preprocessing

Ham veri setinin büyüklüğü ve gerçek dünya operasyonlarından toplanmış olmasından kaynaklanan sistemsiz hataları (anomaliler) ayıklamak adına, veri toplama sürecinin hemen ardına bir ETL (Extract-Transform-Load) boru hattı entegre edilmiştir. Bu doğrultuda uygulanan filtreleme adımları ve teknik gerekçeleri aşağıda sıralanmıştır:

1.2.1 Projection Pruning

Uygulama: Ham veride bulunan 19 kolondan sadece 6 tanesi (tpep_pickup_datetime, tpep_dropoff_datetime, PULocationID, DOLocationID, trip_distance, total_amount) seçilmiş, diğer operasyonel kolonlar elenmiştir.

Gerekçe: Dağıtık sorgu motorlarında disk I/O maliyetini ve bellek (RAM) ayak izini en aza indirmek. Gereksiz verilerin elenmesi, büyük veri setlerinde karşılaşılan OutOfMemoryError (Bellek Yetersizliği)

1.2.2 Zaman Mantığı ve Dönem Doğrulaması

Uygulama: tpep_dropoff_datetime > tpep_pickup_datetime filtresi uygulanmış ve sadece pickup_year == 2023 olan veriler tutulmuştur.

Gerekçe: Sistemsiz senkronizasyon hatalarından dolayı iniş zamanının, binış zamanından önce veya eşit görüldüğü tutarsız kayıtları ayıklamak. Ayrıca donanım kayıtlarındaki bozulmalar nedeniyle geçmiş veya gelecek yıllara aitmiş gibi görünen hatalı tarih girişlerini engellemek.

1.2.3 Seyahat Süresi Filtreleme

Uygulama: Giriş ve çıkış zaman damgaları arasındaki fark alınarak saniye cinsinden seyahat süresi hesaplanmış; 30 saniyeden kısa ve 5 saatten (18.000 saniye) uzun olan yolculuklar elenmiştir.

Gerekçe: 30 saniyenin altındaki kayıtlar genellikle iptal edilen veya yanlışlıkla açılıp kapanan taksimetreleri temsil eder. 5 saatin üzerindeki seyahatler ise sürücünün taksimetreyi açık unutması veya sistem kilitlenmelerinden kaynaklanan anomalilerdir; bu kayıtlar süre tahmin modellerini manipüle ettiği için temizlenmiştir.

1.2.4 Maliyet Filtreleme

Uygulama: Toplam ücretin 0.0\$'dan büyük ve 500.0\$'dan küçük olması koşulu ($0.0 < \text{total_amount} < 500.0$) aranmıştır.

Gerekçe: Negatif veya sıfır değer içeren kayıtlar sistem hatalarını veya iptalleri gösterir. 500\$'ın üzerindeki astronomik rakamlar ise (POS cihazına yanlış veri girilmesi gibi) "fat-finger" olarak adlandırılan insan hataları veya sistemsel kaymalardır. New York şehir içi tarifelerinde bu sınırın üstü istatistiksel olarak aykırı değer (outlier) kabul edilir.

1.2.5 Rota Mesafesi Filtreleme

Uygulama: Yolculuk mesafesinin 0.0 milden büyük ve 100.0 milden küçük olması şartı ($0.0 < \text{trip_distance} < 100.0$) getirilmiştir.

Gerekçe: Sıfır veya negatif mesafeler gerçek bir yolculuğu temsil etmediğinden simülasyonun doğruluğunu bozar. 100 mil üst sınırı ise tüm New York metropolitan alanını ve çevre havalimanlarını güvenli bir şekilde kapsarken, cihaz arızalarından kaynaklanan binlerce millik uçuk hatalı verileri eler.

1.2.6 Coğrafi Bölge Doğrulaması

Uygulama: Biniş ve iniş mahalle kodlarının 1 ile 263 arasında olması kısıtı konulmuştur.

Gerekçe: taxi_zone_lookup.csv referans tablosunda New York'a ait resmi mahalle tanımları 1-263 arasındadır. Veri setinde yer alan 264 ve 265 kodları "Bilinmeyen Bölge" (Unknown / No Zone) anlamına gelir. Kullanıcının haritadan veya arayüzden bilinmeyen bir lokasyonu seçerek simülasyon başlatması mümkün olmayacağından, bu eksik verili satırlar analize dahil edilmemiştir.

2. Kullanılan Platform ve Sistemlerin Durumu

Projenin karşılaştırmalı analiz omurgasını oluşturan iki dağıtık sorgu motoru (Apache Spark ve Apache DataFusion) ile bunları destekleyen çevresel bileşenler kurulmuş, temel çalışabilirlik testlerinden geçirilmiştir. Ekip üyelerinin tamamının aynı sürümlerle çalışmasını sağlamak amacıyla ortak bir requirements.txt dosyası ve GitHub deposu (github.com/nisaklsoglu/NYC-Taxi-Bench) oluşturulmuştur.

2.1 Sorgu Motorları

Apache Spark (PySpark 3.5.1): Ana sorgu motorlarından biri olarak konumlandırılmıştır. Yerel modda (local[*]) çalışacak şekilde yapılandırılmış; sürücü (driver) ve executor bellek sınırları performans testleri için 4 GB olarak ayarlanmıştır. Preprocessing boru hattı ve baseline sorgular Spark üzerinden başarıyla yürütülmüş; JVM tabanlı çalışma modeli, Catalyst optimizasyonu (predicate pushdown, projection pruning) ve DAG tabanlı yürütme özellikleri aktif olarak kullanılmaktadır.

Apache DataFusion (datafusion-python): Karşılaştırmanın diğer ayağını oluşturan Rust tabanlı, JVM bağımlılığı olmayan modern kolonar sorgu motorudur. Python arayüzü pip üzerinden kurulmuş, aynı parquet dosyaları üzerinde ANSI SQL sorgularının çalıştırılabildiği doğrulanmıştır. Karşılaştırmanın adilliği için Spark tarafında oluşturulan temiz veri seti (data/clean/) DataFusion tarafından da doğrudan okunacak biçimde tasarlanmıştır.

2.2 Programlama Dili ve Kütüphaneler

- **Python 3.11:** Ana geliştirme dili. Her ekip üyesinin makinesinde izole bir sanal ortam (venv_401) altında yönetilmektedir; sistemsel Python sürümlerinden bağımsız bir kurulum sağlanarak sürüm çatışması riski ortadan kaldırılmıştır.
- **Java 17 (Eclipse Temurin OpenJDK):** Apache Spark için gerekli JVM bağımlılığı.
- **PyArrow:** Parquet dosyalarının kolonar formatta okunması ve iki motor arasında sıfır kopyalı veri paylaşımının temeli olan Apache Arrow bağlaması. Preprocessing çıktısı Arrow uyumlu parquet olarak yazılmaktadır.
- **Pandas:** İki motorun sonuçlarının birebir örtüştüğünü doğrulamak için küçük ölçekli sonuç DataFrame'lerinin karşılaştırılmasında kullanılacaktır.
- **Matplotlib:** Performans karşılaştırmalarının çubuk grafik, çizgi grafik ve ısı haritası olarak görselleştirilmesinde kullanılacaktır.
- **psutil:** Sorgular sırasında bellek ve CPU kullanımının ölçülmesinde kullanılacaktır.

2.3 Depolama ve Ortam

- **Depolama Formatı:** Hem ham hem de temizlenmiş veri Apache Parquet formatında tutulmaktadır. Kolonar yapı sayesinde her iki motor da yalnızca sorguda ihtiyaç duyulan kolonları okuyabilmekte, bu da disk I/O maliyetini önemli ölçüde düşürmektedir.
- **Bölümlenme (Partitioning):** Temiz veri pickup_year=2023/pickup_month=XX/ hiyerarşisinde bölümlenmiştir. Bu, ay bazlı sorgularda partition pruning'in devreye girmesini ve gereksiz dosyaların hiç okunmamasını sağlar.
- **Geliştirme Ortamı:** Visual Studio Code + Jupyter eklentisi ve doğrudan Jupyter Notebook. Kaynak kontrol için ortak GitHub deposu; yerel dev ortamlar hem Windows hem Linux/macOS üzerinde çalıştırılabilir biçimde kurgulanmıştır.

3. Kurulum Problemleri, Edinilen Bilgi ve Demo Çalıştırmaları

3.1 Karşılaşılan Kurulum Problemleri ve Çözümleri

Kurulum sürecinde teoriden pratiğe geçerken karşılaşılan sistemsel problemler ve bunların çözümleri aşağıda dokümente edilmiştir. Bu problemler, projenin ileri aşamalarında yeni bir ekip üyesinin ortamı hızlıca kurabilmesi için referans niteliği taşımaktadır.

3.1.1 Java Sürüm Uyumsuzluğu

Problem: Ekip üyelerinin bir kısmında sistemde kurulu Java sürümü 22 idi. Apache Spark 3.5 resmi olarak Java 8, 11 ve 17 sürümlerini destekler; Java 22 ile "Unsupported class file major version" tipinde çalışma zamanı hataları alınmıştır.

Çözüm: Eclipse Temurin OpenJDK 17 (LTS) sürümü kurulmuş; JAVA_HOME sistem değişkeni bu yeni kurulumla yönlendirilmiş ve PATH değişkeninin en başına Java 17'nin bin klasörü eklenmiştir. Böylece Java 22 sistemde bırakılmış olmasına rağmen, komut satırından çağrılan java komutu artık Java 17'yi işaretlemektedir.

3.1.2 Windows – winutils.exe Bağımlılığı

Problem: Windows üzerinde Spark başlatıldığında, Hadoop'un yerel dosya sistemi işlemleri için ihtiyaç duyduğu winutils.exe ve hadoop.dll dosyalarının bulunamamasından kaynaklanan "HADOOP_HOME and hadoop.home.dir are unset" hatası alınmıştır.

Çözüm: Hadoop 3.3.5 sürümüne ait uyumlu winutils.exe ve hadoop.dll dosyaları C:\hadoop\bin altına yerleştirilmiş; HADOOP_HOME ortam değişkeni C:\hadoop olarak ayarlanmış ve bu yol PATH değişkenine eklenmiştir. Sorun Linux/macOS ortamlarında zaten gözlemlenmemektedir.

3.1.3 Ortam Değişkeni Yetki Sorunu

Problem: JAVA_HOME ve PATH değişkenleri makine (Machine) seviyesinde ayarlanmak istendiğinde "İstenen kayıt defteri erişimine izin verilmiyor" (Registry access denied) hatası alınmıştır.

Çözüm: PowerShell "Yönetici olarak çalıştır" seçeneği ile başlatılarak ortam değişkenleri Machine seviyesinde başarıyla yazılabilmektedir. Yönetici erişimi olmayan makinelerde alternatif olarak User seviyesinde ayar yapılabildiği doğrulanmış ve dokümente edilmiştir.

3.1.4 Bellek Yönetimi

Problem: İlk denemelerde 12 aylık veri kümesinin (~12 GB) tek seferde okunmaya çalışılması Spark tarafında OutOfMemoryError ile sonuçlanmıştır.

Çözüm: İki katmanlı bir çözüm uygulanmıştır. Birincisi, SparkSession konfigürasyonunda spark.driver.memory=4g ve spark.executor.memory=4g değerleri açıkça verilmiştir. İkincisi ve daha önemlisi, ETL boru hattı ay bazında iteratif olarak yeniden tasarlanmıştır: her ay ayrı okunmakta, temizlenmekte ve partitionBy ile diske append edilmektedir. Bu yaklaşım hem bellek tüketimini sabit tutmakta hem de olası bir hata durumunda süreci baştan başlatma zorunluluğunu ortadan kaldırmaktadır.

3.1.5 Python Sürüm Uyumsuzluğu

Problem: Bazı ekip makinelerinde sistem Python'u 3.14 sürümü idi; ancak PySpark 3.5.1, Py4J bağımlılığı nedeniyle bu sürümle sağlıklı çalışmamaktadır.

Çözüm: Python 3.11 kurulmuş ve proje kök dizini altında yalıtık bir venv_401 sanal ortamı bu sürümle oluşturulmuştur. Böylece sistemsel Python'a dokunulmadan proje için stabil bir çalışma ortamı sağlanmıştır. Ortak requirements.txt ile bağımlılıklar sabitleştirilmiştir. Ayrıca PySpark 3.5.1'in halen Python'ın distutils modülünü çağırdığı; bu modülün ise Python 3.12 ile birlikte standart kütüphaneden kaldırılmış olduğu gözlemlenmiştir. Sorun, geriye dönük uyumluluk sağlayan setuptools paketinin (pip install setuptools) kurulmasıyla çözülmüştür.

3.2 Edinilen Bilgi (Know-How)

Kurulum ve ilk çalıştırmalar sırasında sistemlerin iç işleyişine dair önemli gözlemler edinilmiştir:

- Parquet dosyalarının kolonlar yapısı sayesinde her iki motor da sorguda ihtiyaç duyulmayan kolonları hiç okumamakta; bu, disk I/O maliyetini önemli ölçüde düşürmektedir. Aynı prensip pickup_month ile bölümlenmiş dizinlerde partition pruning aracılığıyla klasör seviyesine kadar genişlemektedir.
- Spark tarafında Catalyst optimizer, DataFrame API kullanıldığında predicate pushdown ve projection pruning işlemlerini otomatik olarak uygulamaktadır. Bu nedenle preprocessing sırasında select ve filter çağrılarının sırası, oluşan fiziksel plan üzerinde belirleyici değildir.
- Spark başlatma maliyeti (JVM ısınması, SparkContext oluşturulması) yüksektir; ancak bu maliyet bir kere ödendikten sonra aynı SparkSession üzerinde çalıştırılan ardışık sorgular çok daha hızlı yürümektedir. Karşılaştırma testlerinde bu "warm-up" etkisinin ayrı raporlanması gerekmektedir.
- DataFusion Rust tabanlı olduğu için JVM ısınma süresine ihtiyaç duymamakta, ilk sorguda dahi düşük gecikmeyle yanıt vermektedir. Bu davranış, küçük ölçekli sorgularda DataFusion'ın avantajını doğrulamaktadır.
- Aynı SQL sorgusunun her iki motorda çalıştırılabilir olması, ANSI SQL uyumlu sorgular yazıldığında karşılaştırmayı büyük ölçüde kolaylaştırmaktadır. Bu, dispatcher katmanının aynı sorgu metnini iki motora yönlendirebilmesinin temelini oluşturmaktadır.
- partitionBy ile diske yazılan veri, sonraki sorgularda yalnızca ilgili aya ait alt klasörün okunmasını sağladığı için ölçüm süreleri gerçek dünya kullanım senaryosunu yansıtmaktadır.
- ANSI SQL uyumluluğunda iki motor arasında ince farklar bulunmaktadır. Spark, karışık büyük/küçük harf içeren kolon adlarını (örn. PULocationID) tırnaksız kabul ederken, DataFusion ANSI SQL standardına daha katı uyduğu için bu tür kolonları çift tırnak içinde beklemektedir ("PULocationID"). Bu, dispatcher katmanının aynı sorgu metnini iki motora yönlendirirken sözdizimsel bir dönüşüm katmanı gerektirebileceğini ortaya koymaktadır ve baseline testleri sırasında somut olarak gözlemlenmiştir.

3.3 Demo Çalıştırmaları

3.3.1 Spark Preprocessing Boru Hattı

İlk büyük ölçekli demo çalıştırma, Bölüm 1'de tanımlanan ETL boru hattının Apache Spark üzerinde 2023 yılının 12 ayı için art arda çalıştırılması olmuştur. Süreç ay bazlı iteratif olarak yürütülmüş ve tamamı hatasız tamamlanmıştır.

Ham veri kümesindeki 38.310.226 satırdan 36.542.807 satır analize uygun bulunmuş; aykırı değer, tutarsız zaman damgası ve geçersiz zone kodu filtreleri sonucunda %4.61 oranında bir eleme gerçekleşmiştir. Bu oran, verinin büyük çoğunluğunun temiz olduğunu ve NYC TLC kaynağının kaliteli olduğunu göstermektedir.

Elenen 1.767.419 satırın kırılımı incelendiğinde, yaklaşık 382.882 kaydın (~%1) doğrudan negatif veya sıfır ücret filtresi tarafından yakalandığı; yalnızca 104 kaydın 2023 yılı dışında bir tarih taşıdığı gözlemlenmiştir. Kalan elenmelerin süre (< 30 sn veya > 5 saat), mesafe (0 veya > 100 mil) ve tekrar eden kayıt filtrelerine dağıldığı belirlenmiştir. Ayrıca ham veri üzerinde yapılan denetimler, PULocationID / DOLocationID kolonlarında null içeren kayıtların da bulunduğunu göstermiştir; bu durum ham verinin 'olduğu gibi' güvenilir olmadığını ve preprocessing katmanının kritik rolünü doğrulamaktadır.

Çıktı, pickup_year=2023/pickup_month=1...12 hiyerarşisinde bölünmüş toplam 408 parquet part-file ve yaklaşık 1.43GB'lık bir dizin olarak yazılmıştır. Temiz verinin boyutu, ham veriye göre eklenen türetilmiş kolonlar (pickup_year, pickup_month, trip_duration_seconds) ve partition başına metadata overhead'i nedeniyle bir miktar büyümüştür.

Bu çalıştırma; Spark ortamının uçtan uca stabil çalıştığını, partitionBy destekli iteratif append yaklaşımının bellek profilini sabit tuttuğunu ve preprocessing filtrelerinin veri kaybı yaratmadan hedeflenen anomalileri ayıkladığını göstermiştir.

3.3.2 Baseline Sorgu Denemesi

Boru hattının doğruluğunu ve temiz veri şemasının beklendiği gibi olduğunu doğrulamak için tek aylık temizlenmiş veri üzerinde bir baseline sorgu çalıştırılmıştır: "2023 Ocak ayında belirli bir mahalle çiftinden yapılan yolculukların ortalama ücreti ve süresi". Sonuç Spark tarafında beklenen aralıkta değerler üretmiş; şema kontrolleri (kolon adları, veri tipleri) ve satır sayısı sağlamaları başarıyla geçilmiştir.

3.3.3 İki Motor Arası Baseline Karşılaştırma

Bölüm 2'de tanımlanan iki motorun hem çalışır durumda hem de doğru sonuç ürettiğini göstermek amacıyla, temizlenmiş Ocak 2023 verisi üzerinde birebir aynı sorgu her iki motorda da çalıştırılmıştır: "En yoğun ilk 5 mahalle çifti için ortalama ücret, ortalama süre ve yolculuk sayısı".

İki motorun ürettiği sonuç tablosu birebir örtüşmüştür; en yoğun rota olarak PULocationID=237, DOLocationID=236 (Upper East Side içi kısa yolculuklar), ortalama 15.43 USD ücret ve 6.98 dakika süre ile tespit edilmiştir. Bu tutarlılık, iki motorun aynı ANSI SQL sorgusundan aynı ilişkisel sonucu ürettiğinin ve karşılaştırmanın adil bir zeminde yürüdüğünün doğrulamasıdır.

Çalışma zamanları açısından ise anlamlı bir fark gözlemlenmiştir:

- **DataFusion: 0.246 saniye**
- **Apache Spark (soğuk cache): 1.841 saniye**

Yaklaşık **7.5 katlık** bu fark, DataFusion'ın Rust tabanlı olması ve JVM ısınma maliyeti taşımamasının doğrudan bir sonucudur. Spark'ın soğuk çalıştırmada içerdiği executor başlatma, plan derleme ve JVM ısınması gibi ek maliyetler, tek atımlık küçük sorgularda DataFusion'a karşı dezavantaj yaratmaktadır. Bu bulgu, dispatcher katmanının küçük hacimli ve izole sorguları DataFusion'a, ardışık ve büyük hacimli iş yüklerini ise Spark'a yönlendirmesi yönündeki tasarımı somut olarak desteklemektedir.

Bu karşılaştırma bir baseline (temel referans) niteliğindedir. Bir sonraki aşamada karşılaştırmalar üç ekseninde genişletilecektir: veri hacmi (aylık – yıllık), sorgu karmaşıklığı (basit filtre – çoklu join ile agregasyon) ve tekrar sayısı (soğuk – sıcak cache).

4. Makale Bölümleri

4.1 Özet

Büyük veri işleme dünyasında Apache Spark on yılı aşkın süredir hem endüstri hem akademi tarafından fiili standart olarak kabul edilmektedir. Ancak son yıllarda Rust tabanlı ve JVM bağımlılığı olmayan Apache DataFusion gibi kolonar sorgu motorları, özellikle tek makine ve orta ölçekli veri senaryolarında ciddi performans avantajları sunmaya başlamıştır. Bu çalışmada, New York City Taxi & Limousine Commission (NYC TLC) tarafından yayımlanan gerçek dünya taksi yolculuk verileri üzerinde çalışan bir "Zaman ve Maliyet Tahminleme" (Cost & Time Estimation) uygulaması geliştirilmektedir. Uygulama; kullanıcıdan alınan "nereden, nereye, ne zaman" bilgisine göre geçmiş yolculuklardan ortalama ücret ve süre tahmini üretmekte; aynı sorguyu hem Apache Spark (PySpark) hem de Apache DataFusion üzerinde çalıştırarak süre, bellek kullanımı ve CPU kullanımı gibi ölçütleri karşılaştırmaktadır. Ek olarak, gelen sorgunun taranacak veri hacmi, join gereksinimi ve agregasyon karmaşıklığı gibi karakteristiklerine göre işi otomatik olarak daha uygun motora yönlendiren kural tabanlı bir dispatcher katmanı önerilmektedir. Ön deneyler, Spark'ın büyük veri kümelerinde ve karmaşık dönüşümlerde üstün geldiğini; DataFusion'ın ise küçük ve orta ölçekli sorgularda daha hızlı yanıt verdiğini işaret etmektedir. Bu bulgular, "tek motor her senaryoda en iyisi değildir" hipotezini destekler niteliktedir.

4.2 İlgili Çalışmalar

Apache Spark ve MapReduce Mirası: Zaharia ve arkadaşlarının 2010 yılında yayımladığı Spark makalesi, MapReduce paradigmasının yinelemeli algoritmalarda yaşadığı yoğun disk I/O sorununa RDD (Resilient Distributed Dataset) tabanlı bellek içi hesaplama modelini önermiştir. Sonraki yıllarda Structured API, Catalyst optimizör ve Tungsten yürütme motoru ile birlikte modern Spark, deklaratif sorgular üzerinde otomatik plan optimizasyonu sağlayan olgun bir çerçeveye dönüşmüştür.

Apache Arrow ve DataFusion: Apache Arrow projesi, kolonar bellek içi bir standart sunarak farklı sistemler arasında sıfır kopyalı veri paylaşımı imkânı vermiştir. DataFusion ise Arrow üzerine Rust ile inşa edilmiş bir sorgu motoru olup, Andy Grove tarafından başlatılmış ve günümüzde Apache üst çatısı

altında geliştirilmektedir. DataFusion; JVM olmadan çalışması, düşük bellek ayak izi ve hızlı başlangıç süresi gibi nedenlerle tek makine ölçeğinde Spark'a alternatif olarak konumlandırılmaktadır.

Kolonar Sorgu Motorları ve Benchmark Çalışmaları: DuckDB, Polars, Presto ve ClickHouse gibi motorların yükselişi, tek makine analitik sorgularının Spark'a mutlaka ihtiyaç duymadığını göstermiştir. TPC-H ve TPC-DS benchmark'ları üzerinde bu motorlar arasında yapılan karşılaştırmalar literatürde mevcuttur; ancak bu çalışmaların büyük kısmı sentetik veri ve sabit sorgu setleri ile yürütülmektedir.

Adaptif Sorgu Yürütme ve Motor Seçimi: Spark'ın Adaptive Query Execution (AQE) özelliği, tek bir motor içinde plan seçimini çalışma zamanında değiştirebilmektedir. Bu çalışma ise iki farklı motor arasında yönlendirme yapan bir dış dispatcher önerdiği için literatürdeki AQE çalışmalarından farklı bir düzlemde konumlanmaktadır.

Bu Çalışmanın Katkısı: Sentetik veri yerine gerçek dünya NYC taksi verisi kullanılması, sabit benchmark sorguları yerine kullanıcı arayüzünden gelen dinamik sorgu kalıplarının denenmesi ve iki motor arasında kural tabanlı bir dispatcher önerilmesi, bu çalışmayı mevcut karşılaştırma çalışmalarından ayıran temel yönlerdir.

4.3 Önerilen Uygulama

Önerilen sistem üç ana katmandan oluşmaktadır: veri katmanı, motor katmanı ve dispatcher katmanı.

4.3.1 Veri Katmanı

NYC TLC parquet dosyaları yerel diskte `pickup_year=YYYY/pickup_month=MM/` hiyerarşisinde bölümlenmiş şekilde tutulmaktadır. `taxi_zone_lookup` küçük referans tablosu her iki motorda da broadcast join için kullanılacaktır. Bölüm 1'de detaylandırılan ETL boru hattı tek seferlik olarak uygulanmakta ve çıkan temiz veri parquet olarak yazılmaktadır. Böylece hem Spark hem DataFusion aynı temiz veri üzerinde çalışarak karşılaştırmının adil olması sağlanmaktadır.

4.3.2 Motor Katmanı

Aynı sorgu iki farklı arayüz üzerinden yazılmaktadır: PySpark DataFrame API ve DataFusion SQL / DataFrame arayüzü. Her iki motor da aynı parquet dosyaları üzerinde çalışmakta; sonuçların tutarlılığı Pandas DataFrame karşılaştırması ile satır satır doğrulanmaktadır. Sorguların birebir aynı iş mantığını ifade ettiğini garanti etmek için ortak bir sorgu şablon dosyası kullanılmakta ve iki motora ait uygulamalar bu şablondan türetilmektedir.

4.3.3 Dispatcher (Yönlendirici) Katmanı

Kullanıcıdan gelen sorgu önce hafif bir analizörden geçirilmekte, taranacak partition sayısı ve sorgu özellikleri (join, groupBy, filtre karmaşıklığı) tahmin edilmektedir. Ardından aşağıdaki kural tabanlı stratejiye göre motor seçimi yapılmaktadır:

- Taranacak veri hacmi ~1 GB'ın altındaysa → DataFusion (düşük başlangıç maliyeti avantajı).
- Taranacak veri hacmi ~5 GB'ın üzerindeyse → Spark (paralel yürütme avantajı).
- Sorgu ağır join veya geniş groupBy içeriyorsa → Spark (paralel shuffle avantajı).
- Sorgu basit filtre ve tek agregasyon içeriyorsa → DataFusion.

- 1–5 GB arasındaki gri bölge sorgularda → önce DataFusion denenir, belirli bir süre eşiği aşılsa Spark'a devredilir (adaptif seçim).

4.3.4 Ölçüm ve Görselleştirme

Her sorgunun çalışma süresi `time.perf_counter()` ile mikro saniye hassasiyetinde ölçülmekte; bellek ve CPU kullanımı psutil ile takip edilmektedir. Sonuçlar CSV olarak kaydedilmekte ve matplotlib ile çubuk grafik, çizgi grafik ve ısı haritası şeklinde görselleştirilecektir. Karşılaştırmalar üç eksenle yapılacaktır: veri hacmi (aylık – yıllık), sorgu karmaşıklığı (basit filtre – çoklu join ile agregasyon) ve tekrarlı çalıştırma (soğuk – sıcak cache).

4.3.5 Beklenen Katkı

Çalışmanın sonunda; (i) hangi sorgu türünün hangi motorda daha hızlı çalıştığına dair somut bir karar tablosu, (ii) bu tabloyu otomatik uygulayan çalışır durumda bir dispatcher prototipi ve (iii) gerçek dünya taksi verisi üzerinde Spark ve DataFusion karşılaştırmasına dair ayrıntılı bir deneysel rapor elde edilmesi hedeflenmektedir.